



Red Hat build of Apache Qpid Proton DotNet 1.0.0

Using Qpid Proton DotNet

Developing an AMQ messaging client using .NET

Red Hat build of Apache Qpid Proton DotNet 1.0.0 Using Qpid Proton DotNet

Developing an AMQ messaging client using .NET

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack[®] Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

This guide describes how to install and use your client with other AMQ components.

Table of Contents

MAKING OPEN SOURCE MORE INCLUSIVE	4
CHAPTER 1. OVERVIEW	5
1.1. KEY FEATURES	5
1.2. SUPPORTED STANDARDS AND PROTOCOLS	5
1.3. SUPPORTED CONFIGURATIONS	5
1.4. TERMS AND CONCEPTS	5
1.5. DOCUMENT CONVENTIONS	6
The sudo command	6
File paths	6
Variable text	6
CHAPTER 2. INSTALLATION	7
2.1. PREREQUISITES	7
2.2. INSTALLING ON RED HAT ENTERPRISE LINUX	7
2.3. INSTALLING ON MICROSOFT WINDOWS	8
2.4. ADDING THE CLIENT TO YOUR .NET APPLICATION	8
2.5. INSTALLING THE EXAMPLES	8
CHAPTER 3. GETTING STARTED	9
3.1. PREREQUISITES	9
3.2. RUNNING HELLOWORLD ON RED HAT ENTERPRISE LINUX	9
3.3. RUNNING HELLO WORLD ON MICROSOFT WINDOWS	9
CHAPTER 4. EXAMPLES	11
4.1. SENDING MESSAGES	11
Running the example	12
4.2. RECEIVING MESSAGES	12
Running the example	13
CHAPTER 5. NETWORK CONNECTIONS	15
5.1. CONNECTION OPTIONS	15
5.1.1. Connection Transport Options	15
5.2. RECONNECT AND FAILOVER	15
CHAPTER 6. SECURITY	16
6.1. CONNECTING WITH A USER AND PASSWORD	16
6.2. CONFIGURING SASL AUTHENTICATION	16
6.3. CONFIGURING AN SSL/TLS TRANSPORT	16
CHAPTER 7. SENDERS AND RECEIVERS	17
7.1. CREATING QUEUES AND TOPICS ON DEMAND	17
7.2. CREATING DURABLE SUBSCRIPTIONS	17
7.3. CREATING SHARED SUBSCRIPTIONS	18
CHAPTER 8. MESSAGE DELIVERY	19
8.1. SENDING MESSAGES	19
8.2. TRACKING SENT MESSAGES	19
8.3. RECEIVING MESSAGES	19
8.4. ACKNOWLEDGING RECEIVED MESSAGES	19
CHAPTER 9. LOGGING	21
9.1. SETTING THE LOG OUTPUT LEVEL	21
9.2. ENABLING PROTOCOL LOGGING	21

CHAPTER 10. INTEROPERABILITY	22
10.1. INTEROPERATING WITH OTHER AMQP CLIENTS	22
10.2. INTEROPERATING WITH RED HAT BUILD OF APACHE QPID JMS	26
JMS message types	26
10.3. CONNECTING TO AMQ BROKER	27
APPENDIX A. MANAGING CERTIFICATES	28
A.1. INSTALLING CERTIFICATE AUTHORITY CERTIFICATES	28
A.2. INSTALLING CLIENT CERTIFICATES	28
A.3. USING CLIENT CERTIFICATES	29
APPENDIX B. USING YOUR SUBSCRIPTION	30
B.1. ACCESSING YOUR ACCOUNT	30
B.2. ACTIVATING A SUBSCRIPTION	30
B.3. DOWNLOADING RELEASE FILES	30
B.4. REGISTERING YOUR SYSTEM FOR PACKAGES	30
APPENDIX C. USING AMQ BROKER WITH THE EXAMPLES	32
C.1. INSTALLING THE BROKER	32
C.2. STARTING THE BROKER	32
C.3. CREATING A QUEUE	32
C.4. STOPPING THE BROKER	33

MAKING OPEN SOURCE MORE INCLUSIVE

Red Hat is committed to replacing problematic language in our code, documentation, and web properties. We are beginning with these four terms: master, slave, blacklist, and whitelist. Because of the enormity of this endeavor, these changes will be implemented gradually over several upcoming releases. For more details, see [our CTO Chris Wright's message](#).

CHAPTER 1. OVERVIEW

Red Hat build of Apache Qpid Proton DotNet is a lightweight AMQP 1.0 library for the .NET platform. It enables you to write .NET applications that send and receive AMQP messages.

Red Hat build of Apache Qpid Proton DotNet is part of AMQ Clients, a suite of messaging libraries supporting multiple languages and platforms. For information on available clients, see [AMQ Clients](#).

1.1. KEY FEATURES

- SSL/TLS for secure communication
- Flexible SASL authentication
- Seamless conversion between AMQP and native data types
- Access to all the features and capabilities of AMQP 1.0
- An integrated development environment with full *IntelliSense* API documentation

1.2. SUPPORTED STANDARDS AND PROTOCOLS

Red Hat build of Apache Qpid Proton DotNet supports the following industry-recognized standards and network protocols:

- Version 1.0 of the [Advanced Message Queueing Protocol](#) (AMQP)
- Versions 1.0, 1.1, 1.2, and 1.3 of the [Transport Layer Security](#) (TLS) protocol, the successor to SSL
- [Simple Authentication and Security Layer](#) (SASL) mechanisms ANONYMOUS, PLAIN, and EXTERNAL
- Modern [TCP](#) with [IPv6](#)

1.3. SUPPORTED CONFIGURATIONS

Refer to [Red Hat AMQ Supported Configurations](#) on the Red Hat Customer Portal for current information regarding Red Hat build of Apache Qpid Proton DotNet supported configurations.

1.4. TERMS AND CONCEPTS

This section introduces the core API entities and describes how they operate together.

Table 1.1. API terms

Entity	Description
Client	A context for creating Connections to Brokers
Connection	A channel for communication between two peers on a network
Session	A context for sending and receiving messages

Entity	Description
Sender	A channel for sending messages to a target
Receiver	A channel for receiving messages from a source
Delivery	A Delivery received from a IReceiver that can be acted upon and contains a message
Message	A mutable holder of application data

Red Hat build of Apache Qpid Proton DotNet sends and receives *Messages*. Messages are transferred to Brokers via *Senders* and from Brokers using *Receivers*. Received Messages are wrapped in a *Delivery* which can be Accepted, Rejected or Released.

Senders and *Receivers* are created using *Connections* which are in turn created using *Clients*. *Sessions* are established over *Connections*.

1.5. DOCUMENT CONVENTIONS

The sudo command

In this document, **sudo** is used for any command that requires root privileges. Exercise caution when using **sudo** because any changes can affect the entire system. For more information about **sudo**, see [Using the sudo command](#).

File paths

In this document, all file paths are valid for Linux, UNIX, and similar operating systems (for example, **/home/andrea**). On Microsoft Windows, you must use the equivalent Windows paths (for example, **C:\Users\andrea**).

Variable text

This document contains code blocks with variables that you must replace with values specific to your environment. Variable text is enclosed in arrow braces and styled as italic monospace. For example, in the following command, replace **<project-dir>** with the value for your environment:

```
$ cd <project-dir>
```

CHAPTER 2. INSTALLATION

This chapter guides you through the steps to install Red Hat build of Apache Qpid Proton DotNet in your environment.

2.1. PREREQUISITES

- You must have a [subscription](#) to access AMQ release files and repositories.
- To use Red Hat build of Apache Qpid Proton DotNet on Red Hat Enterprise Linux, you must install the the .NET 8.0 developer tools. For information, see the [Getting started with .NET on RHEL 9](#) and [Getting started with .NET on RHEL 8](#).
- To build programs using Red Hat build of Apache Qpid Proton DotNet on Microsoft Windows, you must install Visual Studio.

2.2. INSTALLING ON RED HAT ENTERPRISE LINUX

Procedure

1. Open a browser and log in to the Red Hat Customer Portal **Product Downloads** page at <https://access.redhat.com/downloads>
2. Locate the **Red Hat AMQ Clients** entry in the **INTEGRATION AND AUTOMATION** category.
3. Click **Red Hat AMQ Clients** The **Software Downloads** page opens.
4. Download the **amq-qpid-dotnet-1.0.0** .zip file.
5. Use the **unzip** command to extract the file contents into a directory of your choosing.

```
$ unzip amq-qpid-dotnet-1.0.0.zip
```

When you extract the contents of the .zip file, a directory named **amq-dotnet-1.0.0** is created. This is the top-level directory of the installation and is referred to as **<install-dir>** throughout this document.

6. Use a text editor to create the file **\$HOME/.nuget/NuGet/NuGet.Config** and add the following content:

```
<?xml version="1.0" encoding="utf-8"?>
<configuration>
  <packageSources>
    <add key="nuget.org" value="https://api.nuget.org/v3/index.json" protocolVersion="3"/>
    <add key="amq-clients" value="<install-dir>/lib"/>
  </packageSources>
</configuration>
```

If you already have a **NuGet.Config** file, add the **amq-clients** line to it.

Alternatively, you can move the .nupkg file inside the **nupkg** directory to an existing package source location.

2.3. INSTALLING ON MICROSOFT WINDOWS

Procedure

1. Open a browser and log in to the Red Hat Customer Portal **Product Downloads** page at <https://access.redhat.com/downloads>
2. Locate the **Red Hat AMQ Clients** entry in the **INTEGRATION AND AUTOMATION** category.
3. Click **Red Hat AMQ Clients** The **Software Downloads** page opens.
4. Download the **amq-qpid-proton-dotnet-1.0.0** .zip file.
5. Extract the file contents into a directory of your choosing by right-clicking on the zip file and selecting **Extract All**.

When you extract the contents of the .zip file, a directory named **amq-dotnet-1.0.0** is created. This is the top-level directory of the installation and is referred to as **<install-dir>** throughout this document.

2.4. ADDING THE CLIENT TO YOUR .NET APPLICATION

Using the dotnet CLI you can add a reference to the Red Hat build of Apache Qpid Proton DotNet client to your application which will also download release binaries from the Nuget gallery. The following command should be run (with the appropriate version updated) in the location where your project file is saved.

```
dotnet add package Apache.Qpid.Proton.Client --version 1.0.0
```

Following this command your **csproj** file should be updated to contain a reference to the proton-dotnet client library and should look similar to the following example:

```
<ItemGroup>
  <PackageReference Include="Apache.Qpid.Proton.Client" Version="1.0.0" />
</ItemGroup>
```

Users can manually add this reference as well and use the dotnet restore command to fetch the artifacts from the Nuget gallery.

2.5. INSTALLING THE EXAMPLES

1. Use the **git clone** command to clone the source repository to a local directory named **qpid-proton-dotnet**:

```
$ git clone https://github.com/apache/qpid-proton-dotnet.git
```

Change to the **qpid-proton-dotnet** directory and use the git checkout command to checkout the commit associated with this release:

```
$ cd qpid-proton-dotnet
$ git checkout 1.0.0
```

The resulting local directory is referred to as **<source-dir>** in this guide.

CHAPTER 3. GETTING STARTED

This chapter guides you through the steps to set up your environment and run a simple messaging program.

3.1. PREREQUISITES

- You must complete the [installation](#) procedure for your environment.
- You must have an AMQP 1.0 message broker listening for connections on interface **localhost** and port **5672**. It must have anonymous access enabled. For more information, see [Starting the broker](#).
- You must have a queue named **hello-world-example**. For more information, see [Creating a queue](#).
- Alternatively, you can specify above information using following environment variables: **HOST**, **PORT**, **ADDRESS**
- The examples are available from the source or upstream
- Download the *amq-qpidd-proton-dotnet-1.0.0-src* .zip file. When you extract the contents of the .zip file, a directory named **amq-dotnet-1.0.0-src** is created. This is the top-level directory of the installation and is referred to as **<src-install-dir>** throughout this document.

3.2. RUNNING HELLOWORLD ON RED HAT ENTERPRISE LINUX

The Hello World example creates a connection to the broker, sends a message containing a greeting to the **hello-world-example** queue, and receives it back. On success, it prints the received message to the console.

Change to the **<source-dir>/examples/Example.HelloWorld/** and use **dotnet run** to build and execute the program.

```
$ cd <source-dir>/examples/Example.HelloWorld/
$ dotnet run
Received message with body: Hello World
```

3.3. RUNNING HELLO WORLD ON MICROSOFT WINDOWS

The Hello World example creates a connection to the broker, sends a message containing a greeting to the **hello-world-example** queue, and receives it back. On success, it prints the received message to the console.

Procedure

1. Navigate to **<source-dir>** and open the **Proton.sln** solution file in Visual Studio.
2. Select **Build Solution** from the **Build** menu to compile the solution.
3. Open a command prompt window and execute the following commands to send and receive a message:

```
> cd <source-dir>\bin\Debug
> Example.HelloWorld
Received message with body: Hello World
```

CHAPTER 4. EXAMPLES

This chapter demonstrates the use of Red Hat build of Apache Qpid Proton DotNet through example programs.

The examples are available from the source package or upstream, for more details and examples, see the [Qpid Proton DotNet examples](#).

4.1. SENDING MESSAGES

This client program connects to a server using `<serverHost>` and `<serverPort>`, creates a sender for target `<address>`, sends 100 messages containing `String` and exits.

Example: Sending messages

```
using System;
using Apache.Qpid.Proton.Client; 1

namespace Apache.Qpid.Proton.Examples.HelloWorld
{
    class Program
    {
        private static readonly int MessageCount = 100; 2

        static void Main(string[] args)
        {
            string serverHost =
                Environment.GetEnvironmentVariable("HOST") ?? "localhost"; 3
            int serverPort =
                Convert.ToInt32(Environment.GetEnvironmentVariable("PORT") ?? "5672"); 4
            string address =
                Environment.GetEnvironmentVariable("ADDRESS") ?? "send-receive-example";

            IClient client = IClient.Create(); 5

            ConnectionOptions options = new ConnectionOptions(); 6
            options.User = Environment.GetEnvironmentVariable("USER");
            options.Password = Environment.GetEnvironmentVariable("PASSWORD");

            using IConnection connection = client.Connect(serverHost, serverPort, options); 7
            using ISender sender = connection.OpenSender(address); 8

            for (int i = 0; i < MessageCount; ++i)
            {
                IMessage<string> message =
                    IMessage<string>.Create(string.Format("Hello World! [{0}]", i)); 9
                ITracker tracker = sender.Send(message); 10
                tracker.AwaitSettlement(); 11

                Console.WriteLine(string.Format("Sent message to {0}: {1}",
                    sender.Address, message.Body));
            }
        }
    }
}
```

```

    }
}
}

```

- 1 **using Apache.Qpid.Proton.Client;** Imports types defined in the Proton namespace. Proton is defined by a project reference to library file *Proton.Net.dll* and provides all the classes, interfaces, and value types associated with Red Hat build of Apache Qpid Proton DotNet.
- 2 The number of messages to send.
- 3 **serverHost** is the network address of the host or virtual host for the AMQP connection and can be configured by setting the Environment variable 'HOST'.
- 4 **serverPort** is the port on the host that the broker is accepting connections and can be configured by setting the environment variable **PORT**.
- 5 **Client** is the container that can create multiple Connections to a broker.
- 6 **options** is used for various setting, including 'User' and 'Password'. See [Section 5.1, "Connection Options"](#) for more information.
- 7 **connection** is the AMQP Connection to a broker.
- 8 Create a **sender** for transferring messages to the broker.
- 9 In the message send loop a new message is created.
- 10 The message is sent to the broker.
- 11 Wait for the broker to settle the message.

Running the example

To run the example program, compile it and execute it from the command line. For more information, see [Chapter 3, Getting started](#).

```
<source-dir>\bin\Debug>Example.Send
```

4.2. RECEIVING MESSAGES

This client program connects to a server using **<connection-url>**, creates a receiver for source **<address>**, and receives messages until it is terminated or it reaches **<count>** messages.

Example: Receiving messages

```

using System;
using Apache.Qpid.Proton.Client;

namespace Apache.Qpid.Proton.Examples.HelloWorld
{
    class Program
    {
        private static readonly int MessageCount = 100;

        static void Main(string[] args)

```

1

2


```

{
    string serverHost =
        Environment.GetEnvironmentVariable("HOST") ?? "localhost";
    int serverPort =
        Convert.ToInt32(Environment.GetEnvironmentVariable("PORT") ?? "5672");
    string address =
        Environment.GetEnvironmentVariable("ADDRESS") ?? "send-receive-example";

    IClient client = IClient.Create();

    ConnectionOptions options = new ConnectionOptions();
    options.User = Environment.GetEnvironmentVariable("USER");
    options.Password = Environment.GetEnvironmentVariable("PASSWORD");

    using IConnection connection = client.Connect(serverHost, serverPort, options);
    using IReceiver receiver = connection.OpenReceiver(address);

    for (int i = 0; i < MessageCount; ++i)
    {
        IDelivery delivery = receiver.Receive();
        IMessage<object> received = delivery.Message();
        Console.WriteLine("Received message with body: " + received.Body);
    }
}

```

- 1 **using Apache.Qpid.Proton.Client;** Imports types defined in the Proton namespace. Proton is defined by a project reference to library file *Proton.Net.dll* and provides all the classes, interfaces, and value types associated with Red Hat build of Apache Qpid Proton DotNet.
- 2 The number of messages to receive.
- 3 **serverHost** is the network address of the host or virtual host for the AMQP connection and can be configured by setting the Environment variable **HOST**.
- 4 **serverPort** is the port on the host that the broker is accepting connections and can be configured by setting the environment variable **PORT**.
- 5 **Client** is the container that can create multiple Connections to a broker.
- 6 **options** is used for various setting, including 'User' and 'Password'. See [Section 5.1, "Connection Options"](#) for more information.
- 7 **connection** is the AMQP Connection to a broker.
- 8 Create a **receiver** for receiving messages from the broker.
- 9 In the message receive loop a new delivery is received.
- 10 The message is obtained from the **delivery**.

Running the example

To run the example program, compile it and execute it from the command line. For more information, see [Chapter 3, *Getting started*](#).

```
<source-dir>\bin\Debug>Example.Receive
```

CHAPTER 5. NETWORK CONNECTIONS

5.1. CONNECTION OPTIONS

This section describes how to configure how to configure connections.

The *ConnectionOptions* object can be provided to a Client instance when creating a new connection and allows configuration of several different aspects of the resulting Connection instance.

ConnectionOptions can be passed in the connect method on *IClient* and are used to configure

Example: Configuring authentication

```
ConnectionOptions connectionOptions = new ConnectionOptions();
connectionOptions.User = "user"
connectionOptions.Password = "password"
IConnection connection = client.Connect(serverHost, serverPort, connectionOptions);
```

For a definitive list of options refer to [Connection Options](#)

5.1.1. Connection Transport Options

The ConnectionOptions object exposes a set of configuration options for the underlying I/O transport layer known as the TransportOptions which allows for fine grained configuration of network level options.

Example: Configuring transport options

```
ConnectionOptions connectionOptions = new ConnectionOptions();
connectionOptions.TransportOptions.TcpNoDelay = false;
IConnection connection = client.Connect(serverHost, serverPort, connectionOptions);
```

For a definitive list of options refer to [Connection Transport Options](#)

5.2. RECONNECT AND FAILOVER

When creating a new connection it is possible to configure that connection to perform automatic connection recovery.

Example: Configuring transport reconnection and failover

```
ConnectionOptions connectionOptions = new ConnectionOptions();
connectionOptions.ReconnectOptions.ReconnectEnabled = true;
connectionOptions.ReconnectOptions.ReconnectDelay = 30_000;
connectionOptions.ReconnectOptions.AddReconnectLocation(<hostname>, <port>);
IConnection connection = client.Connect(serverHost, serverPort, connectionOptions);
```

For a definitive list of options refer to [Reconnect and failover](#)

CHAPTER 6. SECURITY

6.1. CONNECTING WITH A USER AND PASSWORD

Red Hat build of Apache Qpid Proton DotNet can authenticate connections with a user and password.

To specify the credentials used for authentication, set the **Username** and **Password** fields on the *ConnectionOptions* object

Example: Connecting with a user and password

```
ConnectionOptions connectionOptions = new();
connectionOptions.User = "user";
connectionOptions.Password = "pass";
```

6.2. CONFIGURING SASL AUTHENTICATION

Client connections to remote peers may exchange SASL user name and password credentials. The presence of the *user* field in the connection URI controls this exchange. If *user* is specified then SASL credentials are exchanged; if *user* is absent then the SASL credentials are not exchanged.

Various SASL mechanisms are supported, please see [SASL Reference](#)

6.3. CONFIGURING AN SSL/TLS TRANSPORT

Secure communication with servers is achieved using SSL/TLS. A client may be configured for SSL/TLS Handshake only or for SSL/TLS Handshake and client certificate authentication. See the [Managing Certificates](#) section for more information.



NOTE

[TLS Server Name Indication](#) (SNI) is handled automatically by the client library. However, SNI is signaled only for addresses that use the *amqps* transport scheme where the host is a fully qualified domain name or a host name. SNI is not signaled when the host is a numeric IP address.

CHAPTER 7. SENDERS AND RECEIVERS

The client uses sender and receiver links to represent channels for delivering messages. Senders and receivers are unidirectional, with a source end for the message origin, and a target end for the message destination.

Sources and targets often point to queues or topics on a message broker. Sources are also used to represent subscriptions.

7.1. CREATING QUEUES AND TOPICS ON DEMAND

Some message servers support on-demand creation of queues and topics. When a sender or receiver is attached, the server uses the sender target address or the receiver source address to create a queue or topic with a name matching the address.

The message server typically defaults to creating either a queue (for one-to-one message delivery) or a topic (for one-to-many message delivery). The client can indicate which it prefers by setting the **queue** or **topic** capability on the source or target.

To select queue or topic semantics, follow these steps:

1. Configure your message server for automatic creation of queues and topics. This is often the default configuration.
2. Set either the **queue** or **topic** capability on your sender target or receiver source, as in the examples below.

Example: Sending to a queue created on demand

```
SenderOptions senderOptions = new SenderOptions();
senderOptions.TargetOptions.Capabilities = new String [] {"queue"};
ISender sender = connection.OpenSender(address, senderOptions);
```

Example: Receiving from a topic created on demand

```
ReceiverOptions receiverOptions = new ReceiverOptions();
receiverOptions.SourceOptions.Capabilities = new String [] {"topic"};
using IReceiver receiver = connection.OpenReceiver(address, receiverOptions);
```

7.2. CREATING DURABLE SUBSCRIPTIONS

A durable subscription is a piece of state on the remote server representing a message receiver. Ordinarily, message receivers are discarded when a client closes. However, because durable subscriptions are persistent, clients can detach from them and then re-attach later. Any messages received while detached are available when the client re-attaches.

Durable subscriptions are uniquely identified by combining the client container ID and receiver name to form a subscription ID. These must have stable values so that the subscription can be recovered.

To create a durable subscription, follow these steps:

1. Set the connection container ID to a stable value, such as **client-1**:

```
ClientOptions clientOptions = new();
```

```
clientOptions.Id = "client-1";
IClient client = IClient.Create(clientOptions);
```

2. Configure the receiver to act like a topic subscription.

```
ReceiverOptions receiverOptions = new ReceiverOptions();
receiverOptions.SourceOptions.Capabilities = new String [] {"topic", "shared"};
receiverOptions.SourceOptions.DurabilityMode = DurabilityMode.UnsettledState;
receiverOptions.SourceOptions.ExpiryPolicy = ExpiryPolicy.Never;
using IReceiver receiver = connection.OpenDurableReceiver(address, "sub-1",
receiverOptions);
```

To detach from a subscription, close the receiver.

1. Delete the subscription.

```
receiver.CloseAsync();
```

if you dont close the receiver then the durable subscription will remain.

7.3. CREATING SHARED SUBSCRIPTIONS

A shared subscription is a piece of state on the remote server representing one or more message receivers. Because it is shared, multiple clients can consume from the same stream of messages.

The client configures a shared subscription by setting the **shared** capability on the receiver source.

Shared subscriptions are uniquely identified by combining the client container ID and receiver name to form a subscription ID. These must have stable values so that multiple client processes can locate the same subscription. If the **global** capability is set in addition to **shared**, the receiver name alone is used to identify the subscription.

To create a shared subscription, follow these steps:

1. Set the connection container ID to a stable value, such as **client-1**:

```
ClientOptions clientOptions = new();
clientOptions.Id = "client-1";
IClient client = IClient.Create(clientOptions);
```

2. Configure the receiver for sharing by setting the **shared** capability:

```
ReceiverOptions receiverOptions = new ReceiverOptions();
receiverOptions.SourceOptions.Capabilities = new String [] {"topic", "shared"};
receiverOptions.SourceOptions.DurabilityMode = DurabilityMode.UnsettledState;
receiverOptions.SourceOptions.ExpiryPolicy = ExpiryPolicy.Never;
using IReceiver receiver = connection.OpenDurableReceiver(address, "sub-1",
receiverOptions);
```

CHAPTER 8. MESSAGE DELIVERY

8.1. SENDING MESSAGES

To send a message, create a client, connection and a sender

Example: Sending messages

```

IConnection connection = client.Connect(serverHost, serverPort, options);
SenderOptions senderOptions = new SenderOptions();
ISender sender = connection.OpenSender(address, senderOptions);
IMessage<string> message = IMessage<string>.Create("Hello World!");
ITracker tracker = sender.Send(message);

```

For more information, see the [Send example](#).

8.2. TRACKING SENT MESSAGES

When the message is sent a Tracker is returned which can be used to track the message or settle it if not using auto settlement

Example: waiting for the broker to settle the message

```

tracker.AwaitSettlement();

```

Example: settling the message

```

tracker.Settle();

```

8.3. RECEIVING MESSAGES

To receive a message, create a connection, client and receiver.

Example: Receiving messages

```

IConnection connection = client.Connect(serverHost, serverPort, options);
using IReceiver receiver = connection.OpenReceiver(address);
IDelivery delivery = receiver.Receive();
IMessage<object> received = delivery.Message();

```

The **Receiver.Accept()** call tells the remote peer that the message was received and processed.

For more information, see the [Receive example](#).

8.4. ACKNOWLEDGING RECEIVED MESSAGES

The *IDelivery* object can be used to accept, reject modify the delivery.

Example: Acknowledging received messages

```

----
```

```
delivery.Accept()
```

```
----
```

Example: Rejecting received messages

```
----
```

```
delivery.Reject()
```

```
----
```

Example: Releasing received messages

```
----
```

```
delivery.Release()
```

```
----
```


CHAPTER 9. LOGGING

Logging is important in troubleshooting and debugging. By default logging is turned off. To enable logging, you must set a logging level and provide a delegate function to receive the log messages.

9.1. SETTING THE LOG OUTPUT LEVEL

The library emits log traces at different levels:

- Error
- Warning
- Information
- Verbose

The lowest log level, *Error*, traces only error events and produces the fewest log messages. A higher log level includes all the log levels below it and generates a larger volume of log messages.

```
// Enable Error logs only.
Trace.TraceLevel = TraceLevel.Error
```

```
// Enable Verbose logs. This includes logs at all log levels.
Trace.TraceLevel = TraceLevel.Verbose
```

9.2. ENABLING PROTOCOL LOGGING

The Log level *Frame* is handled differently. Setting trace level *Frame* enables tracing outputs for AMQP protocol headers and frames.

Logical OR must be used to get normal tracing output and AMQP frame tracing at the same time. For example

```
// Enable just AMQP frame tracing
Trace.TraceLevel = TraceLevel.Frame;
```

```
// Enable AMQP Frame logs, and Warning and Error logs
Trace.TraceLevel = TraceLevel.Frame | TraceLevel.Warning;
```

The following code writes AMQP frames to the console.

Example: Logging delegate

```
Trace.TraceLevel = TraceLevel.Frame;
Trace.TraceListener = (f, a) => Console.WriteLine(
    DateTime.Now.ToString("[hh:mm:ss.fff]") + " " + string.Format(f, a));
```

CHAPTER 10. INTEROPERABILITY

This chapter discusses how to use Red Hat build of Apache Qpid Proton DotNet in combination with other AMQ components. For an overview of the compatibility of AMQ components, see the [product introduction](#).

10.1. INTEROPERATING WITH OTHER AMQP CLIENTS

AMQP messages are composed using the [AMQP type system](#). This common format is one of the reasons AMQP clients in different languages are able to interoperate with each other.

When sending messages, Red Hat build of Apache Qpid Proton DotNet automatically converts language-native types to AMQP-encoded data. When receiving messages, the reverse conversion takes place.



NOTE

More information about AMQP types is available at the [interactive type reference](#) maintained by the Apache Qpid project.

Table 10.1. AMQP types

AMQP type	Description
null	An empty value
boolean	A true or false value
char	A single Unicode character
string	A sequence of Unicode characters
binary	A sequence of bytes
byte	A signed 8-bit integer
short	A signed 16-bit integer
int	A signed 32-bit integer
long	A signed 64-bit integer
ubyte	An unsigned 8-bit integer
ushort	An unsigned 16-bit integer
uint	An unsigned 32-bit integer

AMQP type	Description
ulong	An unsigned 64-bit integer
float	A 32-bit floating point number
double	A 64-bit floating point number
array	A sequence of values of a single type
list	A sequence of values of variable type
map	A mapping from distinct keys to values
uuid	A universally unique identifier
symbol	A 7-bit ASCII string from a constrained domain
timestamp	An absolute point in time

Table 10.2. Qpid Proton DotNet types before encoding and after decoding

AMQP type	Qpid Proton DotNet type before encoding	Qpid Proton DotNet type after decoding
null	null	null
boolean	System.Boolean	System.Boolean
char	System.Char	System.Char
string	System.String	System.String
binary	System.Byte[]	System.Byte[]
byte	System.SByte	System.SByte
short	System.Int16	System.Int16
int	System.Int32	System.Int32
long	System.Int64	System.Int64
ubyte	System.Byte	System.Byte

AMQP type	Qpid Proton DotNet type before encoding	Qpid Proton DotNet type after decoding
ushort	System.UInt16	System.UInt16
uint	System.UInt32	System.UInt32
ulong	System.UInt64	System.UInt64
float	System.Single	System.Single
double	System.Double	System.Double
list	Amqp.List	Amqp.List
map	Amqp.Map	Amqp.Map
uuid	System.Guid	System.Guid
symbol	Amqp.Symbol	Amqp.Symbol
timestamp	System.DateTime	System.DateTime

Table 10.3. Qpid Proton DotNet and other AMQ client types (1 of 2)

Qpid Proton DotNet type before encoding	AMQ C++ type	Red Hat build of Rhea type
null	nullptr	null
System.Boolean	bool	boolean
System.Char	wchar_t	number
System.String	std::string	string
System.Byte[]	proton::binary	string
System.SByte	int8_t	number
System.Int16	int16_t	number
System.Int32	int32_t	number
System.Int64	int64_t	number

Qpid Proton DotNet type before encoding	AMQ C++ type	Red Hat build of Rhea type
System.Byte	uint8_t	number
System.UInt16	uint16_t	number
System.UInt32	uint32_t	number
System.UInt64	uint64_t	number
System.Single	float	number
System.Double	double	number
Amqp.List	std::vector	Array
Amqp.Map	std::map	object
System.Guid	proton::uuid	number
Amqp.Symbol	proton::symbol	string
System.DateTime	proton::timestamp	number

Table 10.4. Qpid Proton DotNet and other AMQ client types (2 of 2)

Qpid Proton DotNet type before encoding	Red Hat build of Apache Qpid Proton Python type	
null	None	
System.Boolean	bool	
System.Char	unicode	
System.String	unicode	
System.Byte[]	bytes	
System.SByte	int	
System.Int16	int	
System.Int32	long	

Qpid Proton DotNet type before encoding	Red Hat build of Apache Qpid Proton Python type	
System.Int64	long	
System.Byte	long	
System.UInt16	long	
System.UInt32	long	
System.UInt64	long	
System.Single	float	
System.Double	float	
Amqp.List	list	
Amqp.Map	dict	
System.Guid	-	
Amqp.Symbol	str	
System.DateTime	long	

10.2. INTEROPERATING WITH RED HAT BUILD OF APACHE QPID JMS

AMQP defines a standard mapping to the JMS messaging model. This section discusses the various aspects of that mapping. For more information, see the Red Hat build of Apache Qpid JMS [Interoperability](#) chapter.

JMS message types

Red Hat build of Apache Qpid Proton DotNet provides a single message type whose body type can vary. By contrast, the JMS API uses different message types to represent different kinds of data. The table below indicates how particular body types map to JMS message types.

For more explicit control of the resulting JMS message type, you can set the **x-opt-jms-msg-type** message annotation.

Table 10.5. Qpid Proton DotNet and JMS message types

Qpid Proton DotNet body type	JMS message type
System.String	TextMessage

Qpid Proton DotNet body type	JMS message type
null	TextMessage
System.Byte[]	BytesMessage
Any other type	ObjectMessage

10.3. CONNECTING TO AMQ BROKER

AMQ Broker is designed to interoperate with AMQP 1.0 clients. Check the following to ensure the broker is configured for AMQP messaging:

- Port 5672 in the network firewall is open.
- The AMQ Broker AMQP acceptor is enabled. See [Default acceptor settings](#).
- The necessary addresses are configured on the broker. See [Addresses, Queues, and Topics](#).
- The broker is configured to permit access from your client, and the client is configured to send the required credentials. See [Broker Security](#).

APPENDIX A. MANAGING CERTIFICATES

A.1. INSTALLING CERTIFICATE AUTHORITY CERTIFICATES

SSL/TLS authentication relies on digital certificates issued by trusted Certificate Authorities (CAs). When an SSL/TLS connection is established by a client, the AMQP peer sends a server certificate to the client. This server certificate must be signed by one of the CAs in the client's *Trusted Root Certification Authorities* certificate store.

If the user is creating self-signed certificates for use by AMQ Broker, then the user must create a CA to sign the certificates. Then the user can enable the client SSL/TLS handshake by installing the self-signed CA file **ca.crt**.

1. From an administrator command prompt, run the MMC Certificate Manager plugin, **certmgr.msc**.
2. Expand the **Trusted Root Certification Authorities** folder on the left to expose **Certificates**.
3. Right-click **Certificates** and select **All Tasks** and then **Import**.
4. Click **Next**.
5. Browse to select file **ca.crt**.
6. Click **Next**.
7. Select **Place all certificates in the following store**
8. Select certificate store **Trusted Root Certification Authorities**.
9. Click **Next**.
10. Click **Finish**.

For more information about installing certificates, see [Managing Microsoft Certificate Services and SSL](#).

A.2. INSTALLING CLIENT CERTIFICATES

In order to use SSL/TLS and client certificates, the certificates with the client's private keys must be imported into the proper certificate store on the client system.

1. From an administrator command prompt, run the MMC Certificate Manager plugin, **certmgr.msc**.
2. Expand the **Personal** folder on the left to expose **Certificates**.
3. Right-click **Certificates** and select **All Tasks** and then **Import**.
4. Click **Next**.
5. Click **Browse**.
6. In the file type pulldown, select **Personal Information Exchange (*.pfx;*.p12)**.
7. Select file **client.p12** and click **Open**.

8. Click **Next**.
9. Enter the password for the private key password field. Accept the default import options.
10. Click **Next**.
11. Select **Place all certificates in the following store**
12. Select certificate store **Personal**.
13. Click **Next**.
14. Click **Finish**.

A.3. USING CLIENT CERTIFICATES

Before a client will return a certificate to the broker, the Red Hat build of Apache Qpid Proton DotNet library must be told which certificates to use. The client certificate file **client.crt** is added to the list of certificates to be used during connection.

```
ConnectionOptions connectionOptions = new();
connectionOptions.SslOptions.ClientCertificateCollection.Add(
    X509Certificate.CreateFromCertFile(certfile)
);
```

In this example, **certfile** is the full path to the **client.p12** certificate installed in the *Personal* certificate store.

APPENDIX B. USING YOUR SUBSCRIPTION

AMQ is provided through a software subscription. To manage your subscriptions, access your account at the Red Hat Customer Portal.

B.1. ACCESSING YOUR ACCOUNT

Procedure

1. Go to access.redhat.com.
2. If you do not already have an account, create one.
3. Log in to your account.

B.2. ACTIVATING A SUBSCRIPTION

Procedure

1. Go to access.redhat.com.
2. Navigate to **My Subscriptions**.
3. Navigate to **Activate a subscription** and enter your 16-digit activation number.

B.3. DOWNLOADING RELEASE FILES

To access .zip, .tar.gz, and other release files, use the customer portal to find the relevant files for download. If you are using RPM packages or the Red Hat Maven repository, this step is not required.

Procedure

1. Open a browser and log in to the Red Hat Customer Portal **Product Downloads** page at access.redhat.com/downloads.
2. Locate the **Red Hat AMQ** entries in the **INTEGRATION AND AUTOMATION** category.
3. Select the desired AMQ product. The **Software Downloads** page opens.
4. Click the **Download** link for your component.

B.4. REGISTERING YOUR SYSTEM FOR PACKAGES

To install RPM packages for this product on Red Hat Enterprise Linux, your system must be registered. If you are using downloaded release files, this step is not required.

Procedure

1. Go to access.redhat.com.
2. Navigate to **Registration Assistant**.
3. Select your OS version and continue to the next page.

4. Use the listed command in your system terminal to complete the registration.

For more information about registering your system, see one of the following resources:

- [Red Hat Enterprise Linux 8 - Registering the system and managing subscriptions](#)
- [Red Hat Enterprise Linux 9 - Registering the system and managing subscriptions](#)

APPENDIX C. USING AMQ BROKER WITH THE EXAMPLES

The Red Hat build of Apache Qpid Proton DotNet examples require a running message broker with a queue named **hello-world-example**. Use the procedures below to install and start the broker and define the queue.

C.1. INSTALLING THE BROKER

Follow the instructions in *Getting Started with AMQ Broker* to [install the broker](#) and [create a broker instance](#). Enable anonymous access.

The following procedures refer to the location of the broker instance as **<broker-instance-dir>**.

C.2. STARTING THE BROKER

Procedure

1. Use the **artemis run** command to start the broker.

```
$ <broker-instance-dir>/bin/artemis run
```

2. Check the console output for any critical errors logged during startup. The broker logs **Server is now live** when it is ready.

```
$ example-broker/bin/artemis run
```

$$\begin{array}{l} \wedge \mid \vee \mid _ \setminus \mid _ \setminus \quad \parallel \\ / \setminus \mid \setminus / \mid \mid \mid \mid \mid) \mid _ _ _ \mid \mid _ _ _ _ _ _ \\ / \wedge \setminus \mid M \mid \mid \mid \mid _ < \mid ' _ / _ \setminus \mid // _ \setminus ' _ \mid \\ / _ _ _ \setminus \mid \mid \mid \mid _ \mid \mid \mid \mid) \mid \mid \mid (_ \mid < _ / \mid \\ / / _ \setminus _ \setminus \mid \mid \setminus _ _ \setminus \setminus \mid _ _ / \mid \setminus _ \setminus _ / \mid \setminus \setminus _ \mid \end{array}$$

Red Hat AMQ <version>

```
2020-06-03 12:12:11,807 INFO [org.apache.activemq.artemis.integration.bootstrap]
AMQ101000: Starting ActiveMQ Artemis Server
```

...

```
2020-06-03 12:12:12,336 INFO [org.apache.activemq.artemis.core.server] AMQ221007:
Server is now live
```

...

C.3. CREATING A QUEUE

In a new terminal, use the **artemis queue** command to create a queue named **hello-world-example**.

```
$ <broker-instance-dir>/bin/artemis queue create --name hello-world-example --address hello-world-example --auto-create-address --anycast
```

You are prompted to answer a series of yes or no questions. Answer **N** for no to all of them.

Once the queue is created, the broker is ready for use with the example programs.

C.4. STOPPING THE BROKER

When you are done running the examples, use the **artemis stop** command to stop the broker.

```
$ <broker-instance-dir>/bin/artemis stop
```

Revised on 2026-02-25 11:33:50 UTC